

使用 Cloud Functions for Firebase 建構全端 Dart 應用程式

來源：<https://codelabs.developers.google.com/deploy-dart-on-firebase-functions?hl=zh-tw>

步驟數：11

本指南說明如何將伺服器端 Dart 部署至 Cloud Functions for Firebase

目錄

1. [簡介](#)
2. [必要條件](#)
3. [為什麼要使用 Dart 做為後端語言？](#)
4. [建立 Flutter 應用程式](#)
5. [建立共用套件](#)
6. [設定 Cloud Functions for Firebase](#)
7. [撰寫函式](#)
8. [使用 Firebase 模擬器套件在本機測試](#)
9. [部署至 Firebase](#)
10. [疑難排解](#)
11. [恭喜](#)

步驟 1 · 約 0 分鐘

1. 簡介

在本程式碼研究室中，您將建構多人計數器應用程式，並瞭解如何將 Dart 用於 Flutter 前端和 Firebase 後端。

您也會瞭解如何在應用程式和伺服器之間共用資料模型，不必重複邏輯。

課程內容

- 將共用的商業邏輯擷取至獨立的 Dart 套件。
 - 以 Dart 原生語言編寫及部署 Cloud Functions for Firebase。
 - 運用 Dart 的預先 (AOT) 編譯功能，減少無伺服器冷啟動次數。
 - 使用 Firebase 模擬器套件在本機測試堆疊。
-

步驟 2 · 約 0 分鐘

2. 必要條件

- [Flutter SDK](#) (最新穩定版)。
 - [Firebase CLI](#) (需要 15.15.0 以上版本)。
 - 程式碼編輯器，例如 [Antigravity](#)、Visual Studio Code、IntelliJ 或 Android Studio，並安裝 Dart 和 Flutter 外掛程式。
 - 熟悉 Flutter 和 Firebase 的基本知識。
-

步驟 3 · 約 0 分鐘

3. 為什麼要使用 Dart 做為後端語言？

許多雲端應用程式會使用 Dart 做為前端 UI，並使用 TypeScript、Python 或 Go 等其他語言做為後端。這需要維護兩組不同的資料模型。資料庫結構定義變更時，您必須更新兩個程式碼庫。

注意：在後端使用 Dart，即可結合用戶端 Flutter 的回應式使用者體驗，以及伺服器的安全驗證，不必重複編寫程式碼。

步驟 4 · 約 0 分鐘

4. 建立 Flutter 應用程式

建立標準 Flutter 應用程式：



```
flutter create my_counter
cd my_counter
# Run the app to see the default counter example
flutter run
```

在標準 Flutter 應用程式中，`lib/main.dart` 會在本機處理計數器狀態：



```
int _counter = 0;

void _incrementCounter() {
  setState(() {
    _counter++;
  });
}
```

這個方法適用於本機狀態，但無法擴充至多人遊戲應用程式，因為伺服器必須做為事實來源。為支援多位玩家，我們會在下列步驟中將這項邏輯移至後端。

步驟 5 · 約 0 分鐘

5. 建立共用套件

為避免在前端和後端重複建立模型，請在專案存放區內建立共用的 Dart 套件。Flutter 應用程式和適用於 Firebase 的函式都依附於這個套件。

在 `my_counter` 專案的根目錄中，執行下列指令：

```
mkdir -p packages
cd packages
dart create -t package shared
```

新增依附元件

在 `packages/shared/pubspec.yaml` 中，新增 JSON 序列化工具：

```
dependencies:
  json_annotation: ^4.9.0

dev_dependencies:
  build_runner: ^2.4.9
  json_serializable: ^6.8.0
```

定義共用模型

建立 `packages/shared/lib/src/models.dart`。這個檔案會定義應用程式和伺服器使用的資料結構。

```
import 'package:json_annotation/json_annotation.dart';

part 'models.g.dart';

@JsonSerializable()
class IncrementResponse {
  final bool success;
  final String? message;
  final int? newCount;

  const IncrementResponse({required this.success, this.message, this.newCount});

  factory IncrementResponse.fromJson(Map<String, dynamic> json) =>
    _$IncrementResponseFromJson(json);

  Map<String, dynamic> toJson() => _$IncrementResponseToJson(this);
}

// Store the function name as a constant to ensure consistency between client and server.
const incrementCallable = 'increment';
```

在 `packages/shared/lib/shared.dart` 中匯出模型：

```
library shared;  
  
export 'src/models.dart';
```

在 `packages/shared` 目錄中，執行建構執行器來產生 JSON 序列化程式碼：

```
dart run build_runner build
```

步驟 6 · 約 0 分鐘

6. 設定 Cloud Functions for Firebase

Cloud Functions for Firebase 是一種無伺服器架構，可讓您自動執行後端程式碼，不必管理及擴充自己的伺服器。Dart 非常適合，因為它會預先 (AOT) 編譯為二進位檔，不需要 Node.js 或 Java 等龐大的執行階段環境。這可大幅縮短函式的冷啟動時間。

前往專案根目錄，然後初始化 Cloud Functions for Firebase：

```
cd ../../
firebase experiments:enable dartfunctions
firebase init functions
dart pub add google_cloud_firestore
```

- 系統提示選擇語言時，請選取「Dart」。

連結共用套件

在 `functions/pubspec.yaml` 中，新增共用套件的相對路徑：

```
dependencies:
  firebase_functions:
  google_cloud_firestore:
  shared:
    path: ../packages/shared
```

步驟 7 · 約 0 分鐘

7. 撰寫函式

如要編寫後端邏輯，請開啟 `functions/bin/server.dart`，然後將內容替換為下列程式碼：

```

import 'dart:convert';
import 'package:firebase_functions/firebase_functions.dart';
import 'package:google_cloud_firestore/google_cloud_firestore.dart'
    show FieldValue;
import 'package:shared/shared.dart';

void main(List<String> args) async {
  await fireUp(args, (firebase) {

    // Listen for calls to the http request and name defined in the shared package.
    firebase.https.onRequest(name: incrementCallable, (request) async {

      // In a production app, verify the user with request.auth?.uid here.
      print('Incrementing counter on the server...');

      // Get firestore database instance
      final firestore = firebase.adminApp.firestore();

      // Get a reference to the counter document
      final counterDoc = firestore.collection('counters').doc('global');

      // Get the current snapshot for the count data
      final snapshot = await counterDoc.get();

      // Increment response we will send back
      IncrementResponse incrementResponse;

      // Check for the current count and if the snapshot exists
      if (snapshot.data() case {'count': int value} when snapshot.exists) {
        if (request.method == 'GET') {
          // Get the current result
          incrementResponse = IncrementResponse(
            success: true,
            message: 'Read-only sync complete',
            newCount: value,
          );
        } else if (request.method == 'POST') {
          // Increment count by one
          final step = request.url.queryParameters['step'] as int? ?? 1;
          await counterDoc.update({'count': FieldValue.increment(step)});
          incrementResponse = IncrementResponse(
            success: true,
            message: 'Atomic increment complete',
            newCount: value + step,
          );
        } else {
          throw FailedPreconditionError(
            'only GET and POST requests are allowed',
          );
        }
      } else {
        // Create a new document with a count of 1

```

```
    await counterDoc.set({'count': 1});
    incrementResponse = const IncrementResponse(
      success: true,
      message: 'Cloud-sync complete',
      newCount: 1,
    );
  }

  // Return the response as JSON
  return Response(
    200,
    body: jsonEncode(incrementResponse.toJson()),
    headers: {'Content-Type': 'application/json'},
  );
});
}
```

步驟 8 · 約 0 分鐘

8. 使用 Firebase 模擬器套件在本機測試

您可以在本機執行前端和後端，不必部署。

從專案根目錄啟動 Firebase 模擬器套件：

```
# Enable functions and firestore for the emulators
firebase init emulators
# Start the emulators and optionally open up the Admin UI
firebase emulators:start
```

連結共用套件

在 `pubspec.yaml` 中，新增共用套件的相對路徑，並新增 http 套件：

```
dependencies:
  http: ^1.6.0
  shared:
    path: ../packages/shared
```

在 Flutter 專案中開啟 `lib/main.dart`，並將內容替換為下列程式碼。這個前端程式碼使用的 `IncrementResponse` 類別與後端相同。

```

import 'dart:convert';

import 'package:http/http.dart' as http;
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:shared/shared.dart';

/// Get from emulator output when running or when deploying:
/// ✓ functions[us-central1-increment]: http function initialized
/// (http://127.0.0.1:5001/demo-no-project/us-central1/increment).
const incrementUrl = 'FIREBASE_FUNCTIONS_URL_HERE';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) => MaterialApp(
    debugShowCheckedModeBanner: false,
    theme: ThemeData(useMaterial3: true, colorSchemeSeed: Colors.blue),
    home: const CounterPage(),
  );
}

class CounterPage extends StatefulWidget {
  const CounterPage({super.key});
  @override
  State<CounterPage> createState() => _CounterPageState();
}

class _CounterPageState extends State<CounterPage> {
  int _count = 0;
  bool _loading = false;

  @override
  void initState() {
    super.initState();
    // Fetch the current count
    _increment(readOnly: true).ignore();
  }

  Future<void> _increment({bool readOnly = false}) async {
    setState(() => _loading = true);
    try {
      // Call the Dart function.
      final uri = Uri.parse(incrementUrl);
      final response = readOnly ? await http.get(uri) : await http.post(uri);

      // Parse the response back into the shared Dart object.
      final responseData = jsonDecode(response.body);
    } catch (e) {
      // Handle error
    }
  }
}

```

```

        final incrementResponse = IncrementResponse.fromJson(responseData);

        if (incrementResponse.success) {
            setState(() => _count = incrementResponse.newCount ?? _count);
        }
    } catch (e) {
        print("Error calling function: $e");
    } finally {
        setState(() => _loading = false);
    }
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(title: const Text('Multiplayer Counter')),
        body: Center(
            child: Column(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                    const Text('You have pushed the button this many times:'),
                    Text(
                        '$_count',
                        style: Theme.of(context).textTheme.headlineMedium,
                    ),
                ],
            ),
        ),
        floatingActionButton: FloatingActionButton(
            onPressed: _loading ? null : _increment,
            tooltip: 'Increment',
            child: const Icon(Icons.add),
        ),
    );
}

```

執行 Flutter 應用程式。點選懸浮動作按鈕時，應用程式會呼叫本機 Dart 後端、擷取新計數，並更新 UI。

步驟 9 · 約 0 分鐘

9. 部署至 Firebase

如要部署 Dart 後端，請執行下列指令：

```
firebase deploy --only functions
```

指令執行完畢後，請複製網址，並取代先前在 Flutter 應用程式原始碼中新增的 `FIREBASE_FUNCTIONS_URL_HERE`。

10. 疑難排解

`firebase: command not found`

請確認已安裝 Firebase CLI，且 `PATH` 已更新。您可以使用 npm 安裝：`npm install -g firebase-tools`。

初始化函式範本缺少 Dart

如要讓 Dart 顯示為部署選項清單，並在執行 `firebase init functions` 時建立範本程式碼，請執行 `firebase experiments:enable dartfunctions` 設定實驗旗標。

函式模擬器無法連線

確認你使用的是 `localhost` 和通訊埠 `5001`。如果您在 Android Emulator 上進行測試，裝置不會將 `localhost` 解析為您的主體機器。更新 `main.dart` 中的模擬器設定，改用 `10.0.2.2`。

找不到共用套件

請在 `functions/pubspec.yaml` 中驗證相對路徑。如果資料夾結構與程式碼實驗室不同，請調整 `path:` `../packages/shared` 指向正確的目錄。

我需要使用 `json_serializable` 嗎？

雖然並非必要，但使用 `json_serializable` 可避免手動編寫 `fromJson` 和 `toJson` 方法時發生錯誤。確保前端和後端預期完全相同的資料格式。

11. 恭喜

您已成功建構完整堆疊的 Dart 應用程式。在共用套件中維護資料模型，可確保 API 回應和用戶端 UI 保持同步，並在整個堆疊中使用單一程式設計語言。
